

PRÁCTICA 2: MATEMÁTICA DISCRETA

CURSO 2025-2026

“CRIPTOGRAFÍA”

Autores:

CANDELA TEJEDO RAGA

GABRIELA ROMERO MARTÍN

GP10

Índice de Contenidos

1. INTRODUCCIÓN.....	3
2. MEJORAS DE MODULAR.PY	5
2.1. TEST DE PRIMARIDAD DE MILLER-RABIN	5
2.2. MÉTODO DE FACTORIZACIÓN DE FERMAT.	6
3. REGISTRO DE USUARIOS	8
3.1. DESARROLLO DEL PROGRAMA	8
3.2. GESTIÓN DE ERRORES	8
4. CRIPTOCHAT.....	9
4.1. DESARROLLO DEL PROGRAMA	9
4.2. GESTIÓN DE ERRORES	9
5. PRACTICA EN GRUPO.....	11
5.1. EXPLICACIÓN DEL CÓDIGO.....	11
6. RSA	13
6.1. GENERAR CLAVES.....	13
6.2. PADDING	14
6.3. CODIFICAR Y DESCODIFICAR	14
6.3.1. Proceso.....	14
6.3.2. Unicode	16
6.3.3. Cadenas	16
6.4. ROMPER CLAVES.....	17
6.5. ATAQUES	18
7. FUNCIONES AÑADIDAS.....	19
8. BIBLIOGRAFÍA	21

1. INTRODUCCIÓN

Este documento expone el trabajo de Candela Tejedo Raga y Gabriela Romero Martín, alumnos de la Escuela Técnica Superior de Ingeniería (ICAI) de la Universidad Pontificia Comillas. Dicho trabajo lleva como título “Criptografía” y se ha realizado dentro de la asignatura “Matemática discreta” de 2º de iMAT.

Como dijo Bruce Schneier, criptógrafo y experto en seguridad informática, “la seguridad no es un producto, sino un proceso”. El término de criptografía proviene del griego y quiere decir “escritura secreta”. Se define como el ámbito de la criptología (disciplina que estudia los sistemas de escritura secreta) que se ocupa de las técnicas de codificado destinadas a alterar las representaciones lingüísticas de ciertos mensajes con el fin de hacerlos incomprensibles para receptores no autorizados. Permite que dos personas puedan intercambiarse mensajes secretos de forma segura utilizando canales inseguros.

El único objetivo de la criptografía era conseguir la confidencialidad de los mensajes. La aparición de la informática y el uso masivo de las comunicaciones digitales han producido un número creciente de problemas de seguridad. Debido a esto los objetivos de la criptología se han generalizado de tal manera que esta ha pasado a encargarse del estudio de los algoritmos, protocolos y sistemas que se utilizan para proteger a la información y dotar de seguridad a las comunicaciones y las entidades que se comunican.

La criptografía de hoy en día se basa en el principio de Kerrckhoffs que enuncia que “la seguridad de un criptosistema no debe depender de mantener secreto el algoritmo de cifrado empleado, sino solo de mantener secreta la clave”. De tal modo que el adversario solo desconoce el texto claro y la clave utilizada.

Dentro del ámbito de la criptografía en esta práctica nos centraremos en el criptosistema RSA (Rivest-Shamir-Adleman) que basa su seguridad en la dificultad computacional de resolver el problema de la factorización de números enteros grandes. En este sistema se generarán claves públicas y privadas de tal manera que todo el mundo tiene acceso a las claves públicas de un usuario (son las que permitirán que cualquiera le pueda mandar mensajes cifrados que pueda descryptar) y solo el usuario conoce su clave privada (esto es lo que le permite descryptar los mensajes que le mandan otros usuarios).

El objetivo de un atacante es obtener la clave privada de un usuario u obtener el mensaje original correspondiente a un criptograma, sin necesidad de conocer la clave. A lo largo de esta práctica se pondrán todos estos conceptos en práctica mediante distintos códigos de Python.

Se nos ha pedido desarrollar un chat (mejor dicho, un criptochat) en el que dos usuarios previamente registrados en la base de datos puedan mandarse mensajes codificados y descodificarlos siguiendo los procesos de encriptación y descryptación

de RSA. Para ello se han desarrollado tres módulos distintos. `Registrarusuarios.py` añadirá usuarios a la base de datos del sistema dándoles sus correspondientes claves privadas y públicas, esta información se guardará en una carpeta `Usuarios` en sus respectivos ficheros. El módulo `rsa.py` contiene todas las funciones necesarias para codificar y decodificar mensajes entre usuarios y además dos funciones que permiten atacar a otro usuario y descryptar un texto sin emplear la clave privada de un usuario. Por otro lado, `criptochar.py` es la interfaz central, este programa deberá permitir la comunicación encriptada entre dos usuarios previamente registrados en el sistema. Además de estos se cuenta con el módulo `modular.py` desarrollado para la práctica 1 y sus correcciones añadidas.

El documento está estructurado en 6 apartados. En el apartado primero se ha planteado una introducción general al problema que es la seguridad de comunicación entre usuarios que permite entender la motivación de este. En el apartado tercero cuarto y quinto se detallan las explicaciones y motivaciones de los distintos algoritmos empleados para el desarrollo óptimo de la práctica. Por último, el apartado sexto recoge las referencias bibliográficas que se citan a lo largo del documento.

2. MEJORAS DE MODULAR.PY

En este apartado se muestran las distintas mejoras y ajustes incorporados a la librería de modular.py con el fin de que el programa creado sea lo más eficiente posible.

2.1. Test de primaridad de Miller-rabin

Con el fin de crear una versión más eficiente de la función `es_primo()` programada en la práctica anterior que valora si un número es primo o no devolviendo un resultado booleano, True si el número es primo False de lo contrario, se ha creado la función `posible_primo()`. Esta función cuya salida es la misma que la de la anterior valora si un número es primo o no siguiendo el algoritmo de Miller-Rabin basado en el test de primalidad de Fermat.

Test primalidad de Fermat:

El pequeño teorema de Fermat enuncia lo siguiente: sea p primo. Para todo $a \in \mathbb{Z}$ no divisible por p se cumple que: $a^{(p-1)} \equiv 1(\text{mod } p)$.

A raíz de este también se llega a que: sea p un número primo. Para todo $a \in \mathbb{Z}$:

$$a^p \equiv a(\text{mod } p)$$

Sabiendo esto podemos pensar que para comprobar si un número es primo o no basta con comprobar si se cumplen estas congruencias para distintos $a \in \mathbb{Z}$. Con cuántos más números probemos más fiable será el test y en el momento en el que se encuentre un a para el que el teorema no se cumpla ya sabemos que p no puede ser primo.

Basado en este teorema aparece el test de primalidad de Miller-Rabin. Este dice que: dado un entero impar $n < 2^s$ lo escribimos como $2^s d + 1$ donde s y d son dos enteros positivos y además d es impar. Entonces, diremos que n es probable primo fuerte para una cierta base a si se cumple alguna de estas dos congruencias:

$$a^d \equiv 1(\text{mod } n)$$

$$a^{2^r d} \equiv -1(\text{mod } n)$$

La idea detrás de esta prueba es que cuando n es un primo impar pasa la prueba por el pequeño teorema de Fermat y porque las únicas raíces cuadradas de $1(\text{mod } n)$ son 1 y -1. Demostremos esto último:

Sea x tal que $x^2 \equiv 1(\text{mod } n)$ se n es un número primo las dos soluciones de x deben ser estrictamente $1(\text{mod } n)$, $-1(\text{mod } n)$.

$$x^2 \equiv 1(\text{mod } n) \Rightarrow x^2 - 1 \equiv 0(\text{mod } n) \Rightarrow (x + 1)(x - 1) \equiv 0(\text{mod } n)$$

Esto quiere decir que:

$n \mid (x+1)(x-1)$ Como n es primo entonces $n \mid x+1$ o $n \mid x-1$, es decir $x \equiv 1 \pmod{n}$ o $x \equiv -1 \pmod{n}$. c. q. d

Sabiendo esto tendremos la siguiente secuencia: $a^{2^s d}, a^{2^{s-1} d}, \dots, a^{2d}, a^d$. Cada término de esta secuencia es una raíz cuadrada del término anterior. Ya que el primer término es congruente con 1, el segundo término es una raíz cuadrada de 1 módulo n . Por lo demostrado anteriormente es congruente con 1, -1 módulo n . Si es congruente con -1 hemos acabado, el número es primo. En resumen, buscamos que los números de la secuencia tengan como raíces solo 1 o -1.

Aplicando este razonamiento a la función *posible_primo*($n: \text{int}$) $\rightarrow \text{bool}$:

1. Casos base: si $n < 2$ o $2 \mid n$ sabemos que el número no puede ser primo. Por otro lado, si $n \in (2,3)$, $n \in \mathbb{Z}$ entonces el número es el primo 2 o 3.
2. Buscamos escribir el número como $n = 2^s d + 1$. Iniciamos con $d = n - 1$, $s = 0$ y con un bucle vamos eliminando los factores de 2 de d y añadiéndolos a s . Al acabar tendremos $n - 1 = 2^s d$ siendo d un número impar.
3. Calcularemos $x \equiv 2^d \pmod{n}$ (2^d es el primer elemento de la secuencia comentada anteriormente que vamos a utilizar). Si $x = 1$, $x = -1$ entonces sabemos que se cumple el teorema y por lo tanto el número es primo, devolvemos True.
4. En el caso contrario deberemos ir probando con el resto de los números de la secuencia $a^{2^s d}, a^{2^{s-1} d}, \dots, a^{2d}, a^d$. Vamos evaluando, elevando al cuadrado sucesivamente (en cada vuelta duplicamos el exponente. Si en algún momento el resultado de la congruencia llega a ser -1 cumple la propiedad y por lo tanto podemos decir que el número es primo.
5. Si al acabar el bucle la propiedad nunca se ha llegado a cumplir entonces el número no es primo, se devuelve False.

Eficiencia:

Este algoritmo es una mejora a la prueba de primalidad de Fermat pues este no tiene en cuenta los números pseudoprimos como los números de Carmichael. Estos son números que cumplen el teorema de Fermat, pero sin ser primos. El test de Miller-Rabin, aunque parte de la misma idea examina más a fondo convirtiéndose en una alternativa más fiable.

2.2. Método de factorización de Fermat.

Para esta práctica se ha desarrollado una función que factorice de manera eficiente números con una estructura muy particular. Este algoritmo factoriza números de la forma $n = xy$ tal que x, y son números primos. Para realizar se ha decidido emplear el método de factorización de Fermat. Este se basa en la representación de un número real como la diferencia de dos cuadrados. Para esto debemos saber que todo número impar se puede representar como diferencia de cuadrados, demostrémoslo:

Supongamos que un entero n es diferencia de cuadrados. Entonces

$n = x^2 - y^2 = (x + y)(x - y)$ y n posee una factorización en la que ambos factores son de igual paridad, en particular, si dichos factores son impares el número es impar. Recíprocamente, si n es impar existe una factorización de tipo $n = ab$ con ambos números de igual paridad. Si tomamos $x = \frac{a+b}{2}$, $y = \frac{a-b}{2}$ se tiene que:

$$\begin{aligned} n = x^2 - y^2 &= \frac{(a+b)^2}{4} - \frac{(a-b)^2}{4} = \frac{a^2 + b^2 + 2ab}{4} - \frac{a^2 + b^2 - 2ab}{4} = \\ &= \frac{2ab}{4} - \frac{-2ab}{4} = ab \text{ c. q. d.} \end{aligned}$$

Sabiendo esto nos bastará con encontrar los números a y b para factorizar el número deseado. Para encontrarlos, partiremos desde la raíz del número n y escogeremos un x ligeramente mayor. Después calculamos x^2 y le restamos n y obtenemos un cuadrado (número de la forma y^2) hemos terminado. De lo contrario calculamos $(x + 1)^2$ y vemos si al restarle n obtenemos un cuadrado. Seguimos así hasta encontrar los números que buscamos.

Bien es cierto que este método no es el óptimo en todos los casos para la factorización de un número para esta práctica al estar trabajando con productos de dos primos relativamente grandes si nos va a resultar útil.

Aplicamos esta teoría a nuestra función:

1. Primero eliminamos el caso base. Si n es un número par sabemos que será de la forma $n = 2b$ siendo b el resultado de la división entera del número entre 2 por lo tanto devolvemos esos dos valores. De lo contrario n es impar, continuamos en la función.
2. Empezamos calculando x tal que $x = \sqrt{n}$ y vamos sumando 1 hasta que el número al cuadrado sea mayor o igual a n .
3. Ahora buscamos un número y que cumpla la ecuación. Primero tomamos $|y| = x^2 - n$ y vemos si este es cuadrado. Si lo es hemos terminado. De lo contrario aumentamos el valor de x y repetimos el proceso.

Este método de factorización nos será muy útil pues en el caso en el que nos encontramos siempre definiremos la clave privada n como producto de dos primos y hallar estos dos números con facilidad nos va a permitir hallar las claves privadas de los usuarios de manera óptima (o por lo menos de la manera lo más eficiente posible).

3. REGISTRO DE USUARIOS

En este apartado se presenta una descripción y explicación del código situado en el fichero registrarusuarios.py encargado de añadir usuarios a la base de datos del chat y almacenar las claves públicas y privadas de estos.

3.1. Desarrollo del programa

El programa comenzará por pedirle al usuario los datos necesarios para su registro: su nombre, los valores límite para elegir los primos a partir los cuales se crearán sus claves públicas y privadas y el número de padding necesario para la comunicación con el usuario.

Generaremos las claves empleando la función *generar_claves()* de rsa.py cuyo código se explica más adelante. Después guardamos las claves generadas en dos ficheros correspondientes al usuario, uno para sus claves privadas (al que solo él tendrá acceso) y otro para sus claves públicas (al que todos los usuarios tendrán acceso). Estos dos ficheros estarán a su vez almacenados en una carpeta llamada usuarios, en el código implementado se valida la existencia de esta y se crea en el caso de que no exista.

3.2. Gestión de errores

En el caso en el que no se hayan podido crear las claves públicas o privadas del usuario (ya sea porque los números introducidos no tienen el formato adecuado o porque sus valores son incorrectos, por ejemplo, si el valor mínimo de cada primo es mayor al valor máximo de cada primo) se avisará por la terminal de que no se ha podido registrar al usuario y se le mandará un mensaje especificando el error ocurrido.

4. CRIPTOCHAT

En este apartado se presenta una descripción y explicación del código situado en el fichero `criptochat.py`. Este es la interfaz que van a emplear dos usuarios registrados en la base de datos del programa para comunicarse, a través de esta se mandarán y descifrarán mensajes cifrados uno a otro. La idea es que esta simule la pantalla que ve un individuo (en este caso denominado usuario1) cuando quiere comunicarse con otro (en este caso denominado usuario2).

4.1. Desarrollo del programa

Para empezar, el programa recibirá por argumentos los nombres de los dos usuarios entre los cuales ocurre la comunicación, `usuario1` y `usuario2`. El `usuario1` será el primer nombre que se haya introducido por argumentos, este será el que quiere hablar con `usuario2` (mandarle a dicho usuario un mensaje cifrado) o descifrar un mensaje de `usuario2`. Por ello, al entrar en el programa se le dará a elegir entre tres acciones: cifrar un mensaje para `usuario2` (C), descifrar un mensaje de `usuario2` (D) o salir del programa.

- Cifrar (C): si el usuario elige esta opción el programa le pedirá que escriba un mensaje para `usuario2`, después lo cifrará e imprimirá el mensaje cifrado por pantalla. Para realizar este proceso se emplea la función auxiliar `ac_cifrar` encargada de pedir el mensaje, llamar a función de `rsa.py` encargada de cifrar dicha cadena y finalmente imprimir el resultado, el mensaje cifrado.
- Descifrar (D): si el usuario elige esta opción el programa le pedirá que escriba un mensaje ya codificado que le mandó `usuario2`, después lo descifrar e imprimirá el mensaje solución por pantalla. Para realizar esto se emplea la función auxiliar `ac_descifrar` encargada de pedir el mensaje codificado, llamar a la función de `rsa.py` encargada de descodificarlo y finalmente imprimir la frase resultante por pantalla.
- Salir (S): si el usuario desea cerrar la interfaz de comunicación solo deberá pulsar la S, de este modo se sale del programa.

4.2. Gestión de errores

Para asegurar el correcto funcionamiento del programa se han tomado ciertas decisiones para gestionar los posibles errores que pueden ocurrir. En un primer lugar, se valida si la cantidad y el formato de los argumentos recibidos por pantalla son los adecuados. Estos deben ser dos nombres de usuarios ya registrados en la base de datos, de no ser así se avisará del error por pantalla especificando la razón. Si los nombres de los usuarios introducidos no entran registrados en la base de datos se lanzará una excepción diseñada para este caso, `LecturaError` y se indicará que los usuarios deben registrarse antes de iniciar las comunicaciones.

Por otro lado, también se ha gestionado la posibilidad de que el usuario no introduzca una opción válida, es decir, una opción que no es ni C ni D ni S. Si el usuario introduce esas opciones, pero en minúsculas el programa lo da como válido, sin embargo, si introduce cualquier otra cosa se le avisa de que la opción no se encuentra disponible y se le pide introducir una nueva.

Finalmente, si durante el proceso de encriptación o desencriptación salta un error en los cálculos se le avisa el usuario de esto.

5. PRACTICA EN GRUPO

En este apartado se explica el contenido abordado en la práctica grupal realizada en clase. Esto es una descripción del código desarrollado durante esa sesión y almacenado en el archivo de Python practica_grupo.py.

En esta práctica se nos pedía conseguir cifrar mensajes para enviárselos a otro grupo y al mismo tiempo descifrar mensajes que recibíamos de otros grupos. Finalmente, se nos pidió intentar romper las claves de todos equipos con el fin de conseguir hackearlos y poder descifrar sus mensajes. Para poder realizar esto se ha desarrollado un código que luego nos ha servido de modelo para implementar todas las funciones de la práctica (que están explicadas más adelante en el documento) almacenadas en el fichero rsa.py.

Como conclusión de este trabajo podemos decir que el texto descifrado de X resultó ser un fragmento de la obra El Fractal, escrito por Susana Merchán.

5.1. Explicación del código

Primero creamos las distintas claves públicas y privadas de nuestro usuario y escribimos las públicas en el chat facilitado en el Moodle de la asignatura.

Una vez hecho esto tuvimos que mandar un mensaje cifrado (en este caso el mensaje era simplemente una cadena de números) al grupo sentado en la mesa C. Para ello debimos emplear la fórmula correspondiente y las claves públicas del grupo. Del mismo modo, tuvimos que descifrar un mensaje con la misma estructura enviado por el grupo de la mesa A.

Más adelante, tuvimos que enviar un mensaje al grupo A con la diferencia de que en esta ocasión el mensaje era una cadena de caracteres. Los principios empleados para realizar este proceso son los mismos que se han empleado para la realización de la función encargada de descifrar cadenas de caracteres en el módulo de rsa.py. Del mismo modo tuvimos que descifrar un mensaje con la misma estructura que el anterior enviado por el grupo de la mesa C.

Como última tarea de la práctica tuvimos que romper la clave privada de la mesa A. Al realizar este proceso entendimos el problema que supone la factorización de números grandes y después de esto decidimos optimizar esta función en el fichero de modular.py creando una nueva llamada factorizar_cripto (función ya comentada).

Finalmente, en este módulo se muestra el código que descifra el mensaje de X. Primero empleamos Regex para dividir las distintas letras del mensaje en una lista y después (teniendo en cuenta que previamente hemos encontrado la clave privada del usuario receptor del mensaje) aplicamos la fórmula de descifrar para cada elemento de la lista. Por razones de eficiencia se almacenaban los caracteres ya descifrados en un

diccionario para que, en el caso de que aparecieran de nuevo no tuviéramos que repetir el proceso de descriptación. Finalmente mostramos por pantalla el mensaje solución.

El texto descifrado de X es:

Es de esperar que la familia Chamorro venda el piso de la calle del Pez cuando F ya no pueda subir las escaleras hasta el tercero, o cuando Madrid sea tan ciudad que sea insoportable, y se vayan a vivir al pueblo, o tal vez vayan a Denia en busca de Nino. Con casi toda probabilidad, el fractal creado no se perderá en la mudanza y M tendrá que decidir si confesarlo a la pareja joven que comprará el piso o llevárselo para siempre dentro de ella. Aunque, previsiblemente, el hueco de tres metros cuadrados en la pared de la cocina no desaparezca con la marcha de los Chamorro, queda la diminuta posibilidad de que los habitantes de la casa que hay detrás de la nevera se muden también y en el tercero derecha de la calle del Pez, 34 solo queden los helechos.

Este texto es un fragmento de la obra El fractal, escrito por Susana Merchán

6. RSA

En este apartado se muestra una explicación y descripción detallada de todas las funciones implementadas en el módulo `rsa.py` de la práctica junto con su justificación matemática. Estos códigos nos permiten cifrar y descifrar mensajes empleando el método RSA al igual que romper claves y hackear a otros usuarios.

6.1. Generar claves

Lo primero que se necesita para poder hablar en un chat con encriptación de extremo a extremo es un par de claves, pública y privada, para RSA.

Creamos estas claves con la función `generar_claves()`:

1. Empezamos eligiendo dos primos grandes y distintos (p_1, p_2)
2. Hacemos $n = p_1 p_2$. n es pública, pero nadie sabe tu p_1 y p_2
3. Definimos el módulo aritmético de la clave privada como $\varphi(n) = \varphi(p_1 p_2) = (p_1 - 1)(p_2 - 1)$. Este valor no se publica y es crucial para construir d .
4. Se elige un exponente público e aleatorio que sea coprimo con $\varphi(n)$. Este se publica junto con n . Con (n, e) cualquiera puede cifrar un mensaje privado para ti
5. Se calcula d como la inversa modular de e respecto de $\varphi(n)$. Es la clave secreta y con (n, d) puedes descifrar un mensaje dirigido a ti:

$$ed \equiv 1 \pmod{\varphi(n)}$$

Optimizaciones

- `random.sample()` garantiza $p_1 \neq p_2$ en $O(1)$

Esta función podría trabajar con números de 20 dígitos y generar claves en cuestión de algunos minutos. Sin embargo, cabe decir que este tiempo depende principalmente del tamaño del intervalo introducido por pantalla. Si este intervalo es muy grande elegir los primos p_1 y p_2 puede resultar una tarea muy lenta pues estos se seleccionan de manera aleatoria de una lista de todos los primos dentro del intervalo previamente calculada. Para calcular esta lista debemos valorar si todos los números del intervalo son primos o no y esto puede llegar a ser un proceso extremadamente lento.

6.2. Padding

Si ciframos siempre el mismo mensaje m con la misma clave pública (n, e) , el resultado $c = m^e \bmod n$ es siempre el mismo. Esto permite comparar, construir tablas, etc. Añadir 'ruido controlado' antes de cifrar dificulta los ataques porque:

- Aleatorización $\rightarrow$ un mismo m produce cifrados distintos
- Dificulta correlaciones y ataques por diccionario
- Uniformiza longitudes y evita estructuras obvias en m

Nosotras vamos a 'pegar' k cifras al final de nuestro mensaje usando aritmética decimal:

1. Multiplicamos por 10^k para desplazar los dígitos de nuestro mensaje k posiciones a la izquierda
2. Sumamos un número r entre 0 y $10^k - 1$ (incluidos), que ocupa exactamente esas k posiciones como las nuevas cifras finales (concatenamos k cifras)

Implementamos esto con *aplicar_padding()*:

1. Si no queremos añadir ningún dígito de padding devuelve el mensaje tal cual
2. Calcula r aleatorio
3. Devuelve el mensaje $+r$

Y lo deshacemos con *eliminar_padding()*:

1. Si no habíamos añadido ningún dígito devolvemos el mensaje tal cual
2. Devolvemos la división entera del mensaje entre el número de cifras que se han añadido al cifrar

En ningún momento estamos cambiando el mensaje. Simplemente concatenamos o truncamos cifras a la derecha de cada bit.

6.3. Codificar y decodificar

6.3.1. Proceso

El proceso de cifrado RSA se basa en la aritmética modular. Dado un mensaje m y una clave pública (n, e) :

$$c \equiv m^e \pmod{n}$$

Aplicando esta teoría en *cifrar_rsa()*:

1. Aplica padding decimal al mensaje entero m
2. Realiza el cifrado modular elevando m_{pad} a la potencia e módulo n
3. Devuelve el entero cifrado c

Optimizaciones

- La función *potencia_mod_p()* usa exponenciación modular rápida, por lo que es óptima

El descifrado invierte el proceso de cifrado, utilizando el exponente privado d que cumple la relación:

$$e \cdot d \equiv 1 \pmod{\varphi(n)}$$

Dado el cifrado c , el mensaje original se obtiene como:

$$m_{pad} \equiv c^d \pmod{n}$$

Después, para recuperar el mensaje original m , se elimina el bloque de padding decimal añadido en el cifrado.

El correcto funcionamiento de RSA se basa en el teorema de Euler:

$$m^{e \cdot d} \equiv m \pmod{n}, \quad \text{si } \text{mcd}(m, n) = 1$$

Esto garantiza que aplicar primero e (cifrar) y luego d (descifrar) devuelve el mensaje original.

Implementación en *descifrar_rsa()*:

1. Descifra el bloque
2. Elimina el padding decimal, recuperando así el mensaje original m
3. Devuelve el entero m , correspondiente al punto ed código Unicode del carácter original

Optimizaciones

- La función *potencia_mod_p()* usa exponenciación modular rápida, por lo que es óptima

6.3.2. Unicode

Unicode es el estándar universal para representar texto de todos los idiomas y símbolos en un ordenador.

- Un punto de código se escribe como U+XXXX en hexadecimal

Entonces, podemos usar los puntos de código Unicode para pasar las cadenas de código a carácter y viceversa.

En *codificar_cadena()*:

1. Codifico con comprensión de listas

En *descodificar_cadena()*:

1. Descodificamos validando y usando `"".join(map(chr, m))`

6.3.3. Cadenas

En RSA, el cifrado de un mensaje m se realiza mediante la operación modular:

$$c \equiv m^e \pmod{n}$$

En el caso de cadenas de texto, primero se debe transformar la cadena en una lista de enteros que representen los puntos de código Unicode de cada carácter. A continuación, se cifra individualmente con RSA y se aplica padding decimal.

Aplicando esto a *cifrar_cadena_rsa()*:

1. Codificamos la cadena completa
2. Aplicamos *cifrar_rsa()* a cada punto de código
3. Devuelve una lista de enteros cifrados

Optimizaciones

- La llamada única a *codificar_cadena()* reduce el coste computacional

Por otro lado, el descifrado en RSA consiste en aplicar el exponente privado d a cada bloque cifrado:

$$m \equiv c^d \pmod{n}$$

Una vez recuperado el mensaje con padding, se eliminan las cifras añadidas artificialmente y se obtiene el punto de código original de cada carácter. Finalmente, se reconstruye la cadena Unicode.

Implementación en *descifrar_cadena_rsa()*:

1. Para cada elemento cifrado c en la lista $cList$, calcula el mensaje descifrado
2. Almacena todos los enteros resultantes en una lista
3. Intenta decodificar la lista completa en una sola pasada
4. Si alguno de los valores obtenidos no corresponde a un punto de código Unicode válido, lanza un *ValueError*

Optimizaciones

- Se descifran todos los bloques en una sola lista antes de decodificar, evitando concatenaciones carácter a carácter
- El uso de *try/except* permite interceptar y reportar errores Unicode sin detener el flujo de ejecución

6.4. Romper claves

La seguridad de RSA se basa en la dificultad de factorizar el módulo $n = p_1 p_2$. Para generar la clave privada d , se necesita conocer los factores primos de n , calcular

$$\varphi(n) = (p_1 - 1)(p_2 - 1)$$

y obtener

$$d \equiv e^{-1} \pmod{\varphi(n)}$$

De esta forma se garantiza que:

$$(m^e)^d \equiv m^{ed} \equiv m \pmod{n}$$

Automatizamos este proceso con *romper_clave()*:

1. Factoriza el módulo n , devolviendo $p_1 p_2$
2. Calcula $\varphi(n)$
3. Comprueba que e y $\varphi(n)$ sean coprimos ya que solo en ese caso existe la inversa modular
4. Calcula y devuelve d

Optimizaciones

- Utiliza directamente la factorización para calcular $\varphi(n)$
- Comprueba la condición de coprimidad para evitar excepciones en el cálculo de inversas

Esta función puede romper cualquier clave privada codificada con un n que sea producto de dos primos. En función del tamaño de estos la función tardará más o menos. Haciendo algunas pruebas se ha visto que puede romper claves de más de 20 dígitos en cuestión de escasos segundos pro lo tanto si puede romper cualquier clave dada en el contexto en el que nos encontramos.

6.5. Ataques

Si un sistema cifra carácter a carácter con RSA sin padding y publica (n, e) , es trivial montar un ataque por texto elegido: un atacante puede construir una tabla de cifrados para todos los posibles caracteres del alfabeto y usarla para invertir la función de cifrado por búsqueda. Esto evita tener que factorizar n o calcular la clave privada d (ambas operaciones mucho más caras).

Aplicando esta idea a `ataque_texto_elegido()`:

1. Creamos un diccionario con todos los valores de la tabla ASCII cifrados
2. Para cada código en `cList`, vemos si está en el diccionario, y en ese caso podemos añadir el código descifrado directamente al mensaje
3. Si cualquiera de los códigos no se encuentra en el diccionario, lanza `ValueError` ya que no corresponde con ningún texto plano codificado con RSA

Optimizaciones

- Usar el diccionario permite transformar cada bloque cifrado en $O(1)$, que minimiza el tiempo de descifrado en textos largos
- Usamos `potencia_mod_p` que está optimizada

Esta función puede romper cualquier texto cifrado que no contenga padding. Por ello se ha decidido añadir una optimización explicada a continuación que siga los mismos principios y al mismo tiempo pueda trabajar con el padding.

7. FUNCIONES AÑADIDAS

Siguiendo el apartado anterior de ataques, investigando, llegamos a la conclusión de que si en vez de trabajar con la clave pública (n, e) trabajasemos con (p_1, p_2, e) (una vez factorizado n), podríamos descifrar los mensajes mucho más rápido.

Al descifrar queremos recuperar el mensaje a partir del cifrado con la clave privada (n, d) , donde

$$m \equiv c^d, \quad n = p_1 p_2$$

Por el TCR sabemos que trabajar módulo n es equivalente a trabajar a la vez módulo p_1 y módulo p_2 y luego recombinar.

∴ Reducimos el exponente por Fermat/Euler

- Para primo p , si $\text{mcd}(x, p) = 1$, el Pequeño Teorema de Fermat dice:

$$x^{p-1} \equiv 1 \pmod{p}$$

Y de aquí podemos deducir la reducción del exponente:

$$x^d \equiv x^{d \bmod (p-1)} \pmod{p}$$

Nosotras trabajamos con $x = c \bmod p_i$. Si $p_i | c$, entonces $c \equiv 0 \pmod{p_i}$ y cualquier potencia sigue siendo 0 ∴ la reducción a $d \bmod (p_i - 1)$ es correcta (el resultado sigue siendo 0).

Con los exponentes reducidos:

$$m_{p_i} \equiv c^{d_{p_i}} \pmod{p_i}, \quad i \in \{1, 2\}$$

*Sean m_{p_1} y m_{p_2} los mensajes parciales obtenidos por módulo p_1 y p_2

Tenemos el sistema

$$\begin{cases} x \equiv m_{p_1} \pmod{p_1} \\ x \equiv m_{p_2} \pmod{p_2} \end{cases}$$

Construimos su única solución módulo n :

1. Calculamos la inversa:

$$\text{inv} \equiv (p_2)^{-1} \pmod{p_1}$$

2. Ajustamos:

$$k \equiv (c_{p_1} - c_{p_2}) \text{inv} \pmod{p_1}$$

3. Reconstruimos:

$$c \equiv c_{p_2} + p_2$$

Aplicando esto a `ataque_texto_padding()`:

1. Calculamos d_{p_i}
2. Calculamos la inversa que será necesaria para la recombinación
3. Por cada código en `cList` calculamos m_{p_i} y luego recomponemos el mensaje paddedado
4. Eliminamos el padding de m
5. Intentamos decodificar la cadena, pero lanza `ValueError` si se detecta un código inválido

Optimizaciones

- Trabajar con d_{p_i} (mucho menor que d cuando p_i son grandes) reduce notablemente el coste de la exponenciación modular
- Las potencias sobre p_1, p_2 son mucho más rápidas que sobre n porque los operadores tienen aproximadamente la mitad de los bits
- La recombinación no es cara (simplemente una inversa y unas multiplicaciones)
- Decodificar toda la lista a la vez es más eficiente que decodificar carácter a carácter, y solo en caso de error se localiza el fallo individualmente

8. BIBLIOGRAFÍA

En este apartado se citan los enlaces que se han visitado a lo largo de la realización de la práctica.

- Wikipedia. (s. f.). *Criptografía*. Recuperado de <https://es.wikipedia.org/wiki/Criptograf%C3%ADa> Wikipedia
- Gaussianos. (2007, octubre 5). *La factorización de Fermat*. Recuperado de <https://www.gaussianos.com/la-factorizacion-de-fermat/> Gaussianos
- Oller Marcén, A. M. (2007-2008). *Diferencias de cuadrados* [PDF]. Recuperado de <https://ttm.unizar.es/2007-08/difcuadrados.pdf> ttm.unizar.es
- Rincón Matemático. (s. f.). *Cuadrados – Teoría de números* [Entrada de foro]. Recuperado de <https://foro.rinconmatematico.com/index.php?topic=9911.0> Rincón Matemático
- AcademiaLab. (s. f.). *Prueba de primalidad de Miller-Rabin*. Recuperado de <https://academia-lab.com/enciclopedia/prueba-de-primalidad-de-miller-rabin/> academia-lab.com
- Wikipedia. (s. f.). *Test de primalidad de Miller-Rabin*. Recuperado de https://es.wikipedia.org/wiki/Test_de_primalidad_de_Miller-Rabin Wikipedia
- Reyes-Figueroa, A. (2022, agosto 23). *El Test de Primalidad de Fermat* [PDF]. Recuperado de <https://pfafner.github.io/tn2022/aulas/Aula12.pdf> pfafner
- AcademiaLab. (s. f.). *Prueba de primalidad de Fermat*. Recuperado de <https://academia-lab.com/enciclopedia/prueba-de-primalidad-de-fermat/> academia-lab.com
- Wikipedia. (s. f.). *Test de primalidad de Fermat*. Recuperado de https://es.wikipedia.org/wiki/Test_de_primalidad_de_Fermat Wikipedia
- Wikipedia. (s. f.). *Método de factorización de Fermat*. Recuperado de https://es.wikipedia.org/wiki/M%C3%A9todo_de_factorizaci%C3%B3n_de_Fermat Wikipedia
- Wikipedia. (s. f.). *Diferencia de dos cuadrados*. Recuperado de https://es.wikipedia.org/wiki/Diferencia_de_dos_cuadrados